

Code Audit Report

Host & Organization Management Module

Date of Audit: **June 15, 2026**

Prepared by: **Code Auditor Subagent**

CLEAN CODE SCORE

84%

Overall module quality rating

DEAD CODE ISSUES

14

Unused imports, states & vars

LOGICAL RISKS

4

RLS gaps & catch-block silences

INEFFICIENCIES

2

GET database writes & redundant loops

Executive Summary

This report details a complete static code audit of the **Host & Organization Management** modules, analyzing 7 critical client-side and server-side files. The objective of this analysis is to identify technical debt, obsolete structures, optimization opportunities, and logical risks without modifying database models or business routines. Our findings reveal that while the system behaves as intended on the surface, it carries substantial optimization opportunities, notably: multiple redundant database queries inside and outside transactional blocks, a critical **"Write-on-GET"** performance hazard in the admin preferences API, and several unused state hooks and icon imports causing oversized client payloads.

Detailed Findings - Frontend Modules

src/app/dashboard/organizations/[id]/page.tsx

DEAD CODE

Organization Details Client Dashboard

The details dashboard page for organizations contains multiple unused dependencies and state imports, increasing build footprint and cluttering syntax:

- **Unused Router Hook:** The Next.js useRouter hook is imported and instantiated as `const router = useRouter()` at line 66, but is never referenced.
- **Unused User Context:** The user property is destructured from `useAuth()` on line 67 but is never used anywhere in the JSX layout.
- **Unused Lucide Icons:** Eight imported UI icons are completely unused: `FileText`, `DollarSign`, `Clock`, `Shield`, `Zap`, `CheckCircle`, `ExternalLink`, and `RefreshCw`.
- **Redundant Theme Constants:** `COLORS.success`, `COLORS.warning`, and `COLORS.info` are defined in the file-level theme configuration but are omitted in rendering. The layout implements hardcoded status colors (e.g. `#10b981`, `#f59e0b`) inline instead.

src/app/dashboard/host-management/page.tsx

DEAD CODE

Host Management Client Dashboard

At 4,767 lines, this file serves as a monolithic interface for hosts, intermediaries, and agent commissions, presenting multiple dead code spots:

- **Dead Form State Hooks:** `newContractStart` (line 204), `newContractEnd` (line 205), and `editContractEnd` (line 228) are declared and reset in transaction submissions, but their stored values are never read. Contract terms are dynamically derived from linked property records instead.
- **Unused Declared Variable:** The variable `agentProfileForLandlord` is computed on line 1526 via a `find` lookup on the `agents` list, but is never used inside the landlord profile JSX rendering block.

```
// Line 1526 of page.tsx
if (viewingLandlord) {
  const agentProfileForLandlord = agentsCalculated.find(a => a.id ===
viewingLandlord.id); // Variable is defined but never read

  return (
    <div style={{ display: 'flex', flexDirection: 'column' }}>
      ...
    </div>
  );
}
```

Detailed Findings – API & Server Routes

src/app/api/admin/preferences/route.ts

PERFORMANCE HAZARD

Super Admin Personal Preferences API

A critical architectural anti-pattern exists in the GET handler of the preferences route, executing data updates during read operations:

- **Write-on-GET Anti-pattern:** In order to synchronize API integration keys, the GET endpoint loops over every organization in the database and runs updates if preferences differ. This occurs on **every GET request**, inducing severe database write locks and performance bottlenecks on reading preferences.
- **Redundancy:** This check is fully redundant since the PUT handler (which updates the preferences) already performs this organization-wide synchronization on change.
- **Unused Parameter:** The GET function parameter `request: NextRequest` is declared but never referenced.

```
// GET handler in src/app/api/admin/preferences/route.ts
export async function GET(request: NextRequest) { // Unused parameter
  ...
  if (keysToSync.length > 0) {
    const orgs = await prisma.organization.findMany();
    for (const org of orgs) {
      ...
      await prisma.organization.update({ ... }); // Database WRITE in a GET
    loop!
    }
  }
}
```

src/app/api/dashboard/host-management/route.ts

QUERY INEFFICIENCY / RLS RISK

Host & Intermediary Management API

The host management route contains unused variables, redundant queries, and a potential multi-tenant leakage risk:

- **Unused Variables:** The variable `matchedIds` (mapping matching property UUIDs) is calculated in both the POST and PUT endpoints but is never read or used.
- **Redundant Safe Checks:** In the POST method, if `useExistingHost` is enabled, it queries the database twice to inspect contact records by name, then executes a third duplicate

"Safe check" query for the exact same contact criteria, resulting in redundant query round-trips.

- **Redundant Post-Transaction Fetches:** After running updates inside the `runInTenantContext` transaction block, the code queries the database a second time from outside the block to inspect the contact's `linkStatus`. The database details could have been returned directly from the transaction context.
- **RLS Isolation Gaps:** Direct queries are performed on `prisma.contact` (e.g. lines 457 and 737) outside of the `runInTenantContext` wrapper. In multi-tenant databases relying on Row Level Security, calling `prisma` directly will fail or bypass tenant filters because the session tenant parameters are not set.

`src/app/api/dashboard/host-management/respond-connection/route.ts`

Inter-Organization Connection API

LOGICAL DETAIL

This API manages partner connections and property imports. It exhibits a minor notification logical limitation:

- **Single Admin Notification Fallback:** Upon connection approval or rejection, the route notifies the partner organization's members. It fetches the recipient using `prisma.organizationMember.findFirst({ where: { organizationId: fromOrgId, isActive: true } })`. In organizations with multiple admins, only the first active admin returned receives the notification, leaving others uninformed.

`src/app/api/admin/organizations/[id]/route.ts`

UNUSED PARAMETER

Admin Tenant Management API

A minor clean-up finding in the admin tenant handler:

- **Unused Parameters:** The GET handler defines `request: Request` as its first parameter but never references it. Since Next.js 15 uses asynchronous request parameter mapping, this parameter should either be omitted or replaced with an underscore (`_request`).

Recommended Refactorings

TARGET FILE	TYPE	ISSUE / IMPACT	ACTION PLAN
organizations/[id]/page.tsx	Client UI	Unused imports (icons, useRouter, user) bloating client payload.	Remove the unused Lucide imports, delete the router declaration, and clean up unused fields from useAuth.
host-management/page.tsx	Client UI	Monolithic file (4,767 lines) with multiple unused state variables.	Remove dead form states (newContractStart, newContractEnd, editContractEnd). Extract sub-components for the Landlord and Agent dashboards.
preferences/route.ts	API Route	High Risk: Write-on-GET triggers redundant DB updates on preferences fetch.	Completely remove the self-healing organization loop in the GET handler. Let the PUT handler manage all integration updates.
host-management/route.ts	API Route	Redundant SQL SELECT queries and direct RLS-bypassing queries on <code>prisma.contact</code> .	Return updated entities directly from the transaction. Wrap all database calls to <code>prisma.contact</code> inside the tenant context helper. Remove <code>matchedIds</code> .
respond-connection/route.ts	API Route	Under-notifying team members of connection changes.	Query all active admins/managers of the target organization and create notifications in a batch loop.

